# Smart Home Controller

## Mixed-Voltage Hardware Interfacing and Serial Communication

### Level 1 Proof of Concept (PoC)

**Author:** Project 2 Group

**Institution:** Higher Institute of Computer Science and Information Technology – El Shorouk Academy

**Supervisor:** TA. Ahmed Mahfouz

**Development Environment:** Windows 11, Visual Studio Code

## Abstract

This document presents the design, theoretical background, and implementation of a Level 1 Proof of Concept (PoC) for a Smart Home Controller. The system demonstrates the integration between high-level software control using Python and low-level hardware switching circuits.

The project focuses on safe mixed-voltage hardware interfacing, asynchronous serial communication, and protection mechanisms required when controlling inductive loads such as DC motors. The prototype validates the ability to control physical hardware through wireless commands transmitted from a mobile device.

## 1. Introduction

Smart home automation systems require reliable interaction between software control logic and physical electronic hardware. This project demonstrates a foundational prototype capable of controlling a physical actuator (DC motor) through wireless commands transmitted from a mobile device and processed by a computer-based controller.

The objective of this Proof of Concept is to demonstrate:

- Safe hardware control through software logic
- Reliable Bluetooth-based wireless communication
- Modular system architecture suitable for migration to embedded microcontroller platforms

This system represents the first development stage of a larger embedded smart home automation platform.

## 2. System Architecture Overview

The system is organized into four functional layers:

| Layer | Description |
|---|---|
| User Interface Layer | Android smartphone running a Bluetooth terminal application |
| Wireless Communication Layer | HC-06 Bluetooth module receiving commands |
| Control Logic Layer | Python program running on a PC |

| Layer | Description |
|---|---|
| Hardware Actuation Layer | Transistor switching circuit controlling the DC motor |

### High-Level Architecture

```
[ Android Phone ]
|
| Bluetooth Commands
v
[ HC-06 Bluetooth Module ]
|
| UART Serial Communication
v
[ USB-to-TTL Adapter ]
|
| COM Port
v
[ Python Control Logic ]
|
| Logic Signal
v
[ BJT Transistor Switch ] ---> [ DC Motor + External Battery ]
|
v
[ LED Indicator ]
```

# 3. Theoretical Framework

**Note:** The transistor and diode concepts and theory were discussed before during the academic lectures by our colleagues: **Tarek Ashraf** and **Sondos Mohammed**.

## 3.1 Low-Side Switching Using a BJT Transistor

To control a high-current load such as a DC motor using a low-current logic signal, a 2N2222A NPN Bipolar Junction Transistor (BJT) is used as a low-side switch.

The transistor is placed between the load and ground. When a base current is supplied from the logic controller, it allows a larger collector-emitter current to flow, activating the motor while protecting the logic interface from high current loads.

## 3.2 Inductive Kickback Protection (Flyback Diode)

DC motors behave as inductive loads. When the transistor turns off, the collapsing magnetic field generates a high reverse voltage spike known as back electromotive force (Back-EMF).

A flyback diode is connected in parallel with the motor to provide a safe path for this induced current, preventing damage to the transistor and control circuitry.

### 3.3 Motor Stiction and Kick-Start Capacitance

Electric motors experience static friction (stiction), meaning a higher current is required to start motion than to maintain rotation. A capacitor is added to the circuit to provide a short burst of current during startup, helping the motor overcome initial inertia.

### 3.4 Power Isolation Strategy

To protect the host computer and USB interface, the system uses separate power domains:

| Power Domain | Purpose |
|---|---|
| Logic Power (5V) | Powers USB-TTL interface and LEDs |
| Load Power (7.4V) | Powers the DC motor |

Both domains share a common ground reference to ensure correct signal interpretation while preventing high current spikes from affecting the USB interface.

### 3.5 Asynchronous Serial Communication

Communication between the PC, USB-to-TTL converters, and the Bluetooth module is implemented using the UART protocol. A standard baud rate (typically 9600 bps) is used to ensure reliable communication between all devices.

## 4. Hardware Design

### 4.1 Hardware Components

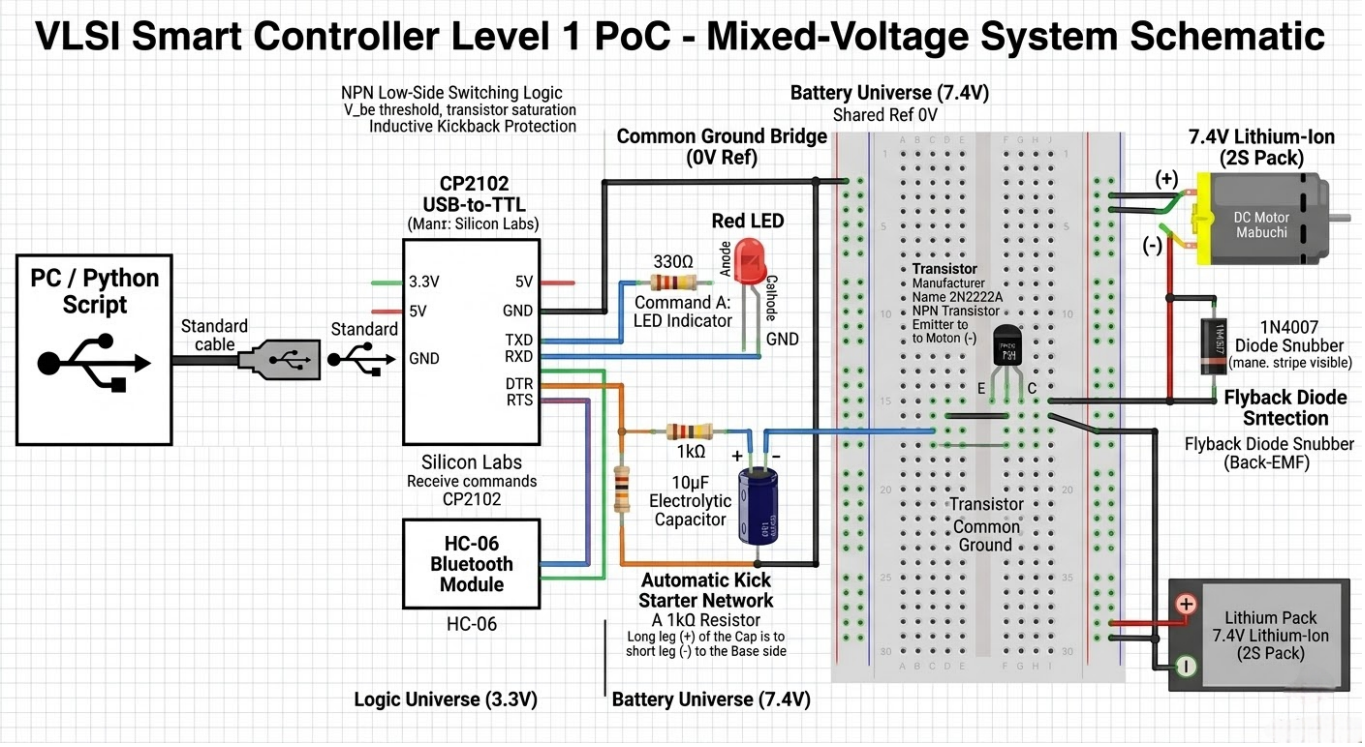
The prototype system includes the following components:

- Two USB-to-TTL serial modules (CP210x)
- HC-06 Bluetooth communication module
- 7.4V lithium-ion battery pack (2S configuration)
- 2N2222A NPN transistor
- Flyback protection diode (1N4007)
- Capacitor for motor kick-start assistance (10μF)
- Assorted resistors for current limiting (1kΩ, 330Ω)
- DC motor as the primary load
- LED indicator
- Breadboard and jumper wires

### 4.2 System Schematic

The following schematic illustrates the mixed-voltage hardware architecture used in the prototype.

**Figure 1:** Mixed-Voltage Smart Controller Level 1 Proof-of-Concept hardware schematic showing the PC control interface, CP2102 USB-to-TTL bridge, HC-06 Bluetooth module, transistor switching stage, and isolated battery-powered motor circuit.




### 4.3 Conceptual Switching Circuit

```
\+7.4V Battery
     |
   \[Motor\]
     |
     \+----|\>|----+
     |   Flyback |
     |    Diode  |
     |           |
     C|
\[2N2222 Transistor\]
     E|
     |
    GND
```

Base Signal --> [Resistor] --> Base

## 5. Software Architecture

### 5.1 Development Environment

| Component | Technology |
|---|---|
| Operating System | Windows 11 |
| IDE | Visual Studio Code |
| Programming Language | Python |

| Component | Technology |
|---|---|
| Serial Library | pyserial |

### 5.2 Supporting Tools

- **PuTTY** – Serial debugging and AT command configuration
- **Serial Bluetooth Terminal (Android)** – Wireless command transmission
- **CP210x Drivers** – USB-to-TTL communication support

### 5.3 Serial Port Configuration

| Port | Function |
|---|---|
| COM5 | Logic control interface |
| COM7 | Bluetooth communication interface |

## 6. System Operation Workflow

1. The user sends a command from the Android Bluetooth terminal.
2. The HC-06 module receives the command via Bluetooth.
3. The command is forwarded through the USB-to-TTL adapter.
4. The Python script reads incoming data through the COM port.
5. The control logic evaluates the command.
6. A control signal is sent to the transistor.
7. The transistor switches the motor circuit.
8. The LED indicator reflects the system status.

## 7. Safety and Protection Mechanisms

The system includes multiple protection mechanisms:

- **Flyback diode** protection against inductive voltage spikes.
- **External battery isolation** from USB power to protect the PC.
- **Current-limiting resistors** for transistor base and LED indicators.
- **Capacitor-assisted startup** to help overcome motor stiction.

## 8. Future Development (Level 2 Architecture)

Future development will migrate the system to an embedded microcontroller platform to create a standalone device.

Planned improvements include:

- Replacement of the PC controller with an **ATmega128A** microcontroller.
- Direct UART communication between the HC-06 module and the microcontroller.
- Support for multiple smart home devices (Lights, Fans, Relays).
- Integration of higher-power MOSFET drivers.
- Development of a **dedicated mobile application** for enhanced UI/UX.

## 9. Project Repository

The complete project source code and documentation are available on GitHub:

https://github.com/AnasMahmoud007/Smart-Home-Controller

The repository includes:

- Python control scripts
- Hardware schematics and diagrams
- Serial communication testing tools
- Future embedded firmware updates

## 10. Conclusion

The Level 1 Proof of Concept demonstrates the successful integration of software-based control logic with hardware switching circuits. The system validates mixed-voltage hardware control, reliable Bluetooth communication, and safe handling of inductive loads.

The prototype establishes a strong foundation for developing a fully embedded smart home automation platform.